

MEDIATEK

MT8668 Hypervisor Performance User Manual

版本: 1.1
出版日期: 2026-02-25

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	黄培俊	正式版
1.1	2026-02-25	黄培俊	移除虚拟机名称相关的信息

目录

版本记录 2

目录 3

1 概述 5

1.1 基本概述 5

2 CPU 状态查看 6

2.1 基本概述 6

2.2 查看 CPU online 的情况 6

2.3 查看 CPU paused 的情况 6

2.4 查看频率的情况 6

3 Cgroup 的信息查询 (Android 端) 7

3.1 基本概况 7

3.2 如何查看各个 process 的 Cgroup 和 sched 信息 7

3.3 如何查看各个 Cgroup 的 cpuset 范围 7

3.4 如何查看各个 Cgroup 的 unclamp 设置情况 7

3.5 如何查看各个 Cgroup 的 latency sensitive 情况 7

4 系统 performance 问题查看 8

4.1 基本概述 8

4.2 Top 信息 8

4.3 中断的问题 8

4.4 内存问题 8

4.5 线程问题 9

4.6 Boost 问题 10

4.7 Trace 分析 10

5 卡顿，黑屏，花屏问题 11

5.1 hwc CMD 11

5.2 显示驱动 CMD 11

5.3 Pattern 12

5.4 黑屏 12

5.5 花屏/闪屏 12

5.6 卡顿 13

5.7 Surfaceflinger Issue Log..... 13

6 算力优化.....15

6.1 Log 的优化..... 15

6.2 常用优化策略..... 15

6.3 异常中断的算力占用..... 16

6.4 多维观察法..... 16

6.5 联动观察法..... 17

6.6 动静态观察法..... 17

附件一 附加条款.....18

1 概述

1.1 基本概述

本章节主要介绍 MT8668 的 CPU 基本能力：4 x B + 4 x L(采用 4 大+ 4 小的结构)。

0	1	2	3	4	5	6	7
小核	小核	小核	小核	大核	大核	大核	大核

2 CPU 状态查看

2.1 基本概述

本章将介绍，如何查看 CPU 的在线情况，paused 情况以及 CPU 运行的频率。

2.2 查看 CPU online 的情况

使用命令：

```
cat /sys/devices/system/cpu/online
```

2.3 查看 CPU paused 的情况

使用命令查看 CPU paused 的情况：

```
cat /sys/devices/system/cpu/sched_ctl/sched_core_pause_info
```

如果是 0x0 就表示 CPU 没有被 paused，如果是非 0 就是表示模块 pause 了 CPU。

2.4 查看频率的情况

1. 查看 CPU 的最大最小频率的情况：

```
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_min_freq
cat /sys/devices/system/cpu/cpufreq/policy4/scaling_min_freq
cat /sys/devices/system/cpu/cpufreq/policy7/scaling_min_freq
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_max_freq
cat /sys/devices/system/cpu/cpufreq/policy4/scaling_max_freq
cat /sys/devices/system/cpu/cpufreq/policy7/scaling_max_freq
```

2. 查看当前各个 CPU 频率的档位：

```
cat /sys/devices/system/cpu/cpufreq/policy0/ scaling_available_frequencies
cat /sys/devices/system/cpu/cpufreq/policy4/ scaling_available_frequencies
cat /sys/devices/system/cpu/cpufreq/policy7/ scaling_available_frequencies
```

3. 查看当前 CPU 的频率方法：

```
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_cur_freq
cat /sys/devices/system/cpu/cpufreq/policy4/scaling_cur_freq
cat /sys/devices/system/cpu/cpufreq/policy7/scaling_cur_freq
```

3 Cgroup 的信息查询 (Android 端)

3.1 基本概况

本章将介绍，Android 端如何查看 Cgroup 的设定和相关信息。

3.2 如何查看各个 process 的 Cgroup 和 sched 信息

1. Process Cgroup 的信息:

```
cat /proc/${pid}/cgroup
```

2. Process sched 的信息:

```
cat /proc/${pid}/sched
```

3. 查看 cpuset 和 cpuctl 下面的 Cgroup 下面的所有 task:

```
cat /dev/cpuset/${cgroup}/tasks  
cat /dev/cpuctl/${cgroup}/tasks
```

3.3 如何查看各个 Cgroup 的 cpuset 范围

可以使用下面的命令获取各个 Cgroup 的 CPU 使用范围:

```
cat /dev/cpuset/${cgroup}/cpus
```

3.4 如何查看各个 Cgroup 的 unclamp 设置情况

可以使用下面的命令获取各个 Cgroup 的 unclamp 设定情况:

```
cat /dev/cpuctl/${cgroup}/cpu.uclamp.min  
cat /dev/cpuctl/${cgroup}/cpu.uclamp.max
```

3.5 如何查看各个 Cgroup 的 latency sensitive 情况

可以使用下面的命令获取各个 Cgroup 的 latency sensitive 设定情况:

```
cat /dev/cpuctl/${cgroup}/cpu.uclamp.latency_sensitive
```

这个设定是设置 perfer idle 属性的。

4 系统 performance 问题查看

4.1 基本概述

本章将介绍跟系统 performance 相关的各种命令和 log。

4.2 Top 信息

Linux 自带的 top

使用方法跟单系统一样，我们一般会观察 top 中 idle 的结果来判断系统的 loading 的情况。

注意：在 Android 端看 idle 的时候要减去 Android 关掉的核心数量，例如现在 Android 关掉了 3 个核，那么实际看 idle 的时候要减去 300%。

4.3 中断的问题

1. 查看系统中断情况

- 总的中断分布：

```
cat /proc/interrupts
```

- 中断的绑核情况：

```
cat /proc/irq/${irq_num}/smp_affinity
```

2. irq monitor 的使用

irq monitor 是 MediaTek 监测 irq 使用异常的工具，默认在 userdebug 和 eng 平台上面打开，user 版本不开。在开启之后默认会监测系统中 irq 执行超时的 case，在监测到系统中有中断处理时间超过 5ms 的情况，就会在 kernel log 里面打印下面的 log：

```
[ 1253.661847] sh: irq_monitor: [name:irq_monitor&]hrtimer:
[<ffffffdcea0cba10>]irq_mon_hrtimer_func.cfi_jt+0x0/0x8 [irq_monitor], duration 600 ms, from
1253061633832 ns to 1253661730757 ns on CPU:2
```

4.4 内存问题

1. Memory 信息的查看

```
cat /proc/meminfo
dumpsys meminfo
```

2. 内存不足关键信息

- 内存不足的 process 信息：
 - Kswapd0：表示内存不足，系统真正通过 kswapd0 大量的回收内存
 - kcompacd0：表示内存碎片很多，系统正在通过 kcompacd0 规整内存碎片
- Android 上层回收内存机制：
 - Lmkd：Android 上层进行内存回收的 process，可以在 main log 里面得到相关的信息
 - HeapTaskDaemon：辅助 GC 的回收，一般可以在 trace 中观察到相关的 thread

4.5 线程问题

1. RT throttle

- RT throttle 的原理

因为 RT 线程的优先级比较高，scheduler 为了不让其他线程饿死（长时间得不到调度），设置了 throttle 机制。即一段时间(一个周期)内所有的 RT 线程执行时间总和超过某个阈值，触发 RT throttle 机制。

周期的长度可以从如下的文件节点读出：

```
cat /proc/sys/kernel/sched_rt_period_us
```

默认为：1000ms

一个周期内允许 running 的最长时间为：

```
cat /proc/sys/kernel/sched_rt_runtime_us
```

默认为：950ms

- 问题识别

当发生 RT throttle 会在 kernel log 打印如下的 log：

```
[ 583.696433] [C1001824] irq/510-1.nebul: sched: RT throttling activated for cpu 0
[ 583.696440] [C1001824] irq/510-1.nebul: sched: cpu=0, expires=584024000000
now=583649109573 rt_time=4140666235 runtime=950000000 period=1000000000
[ 583.696445] [C1001824] irq/510-1.nebul: cpu=0, current irq/510-1.nebul (1824) is
running for 5991740014 nsec
```

2. workqueue lockup

- workqueue lockup 原理

“workqueue lockup”是指工作队列长时间无法完成当前正在运行的工作。可能存在多种原因，例如：

1.当前正在处理的工作太多。

2.由于 CPU 被更高优先级的任务占用，工作队列 kthread 未被调度。

3.由于主机过载，虚拟机没有足够的时间运行。

问题识别

当出现 workqueue lockup 时会出现下面的 log：

```
[ 3450.381929] [ C0] BUG: workqueue lockup - pool cpus=0 node=0 flags=0x0 nice=0
stuck for 3442s!
```

3. RCU stall

RCU stall 的原理

RCU (Read-Copy-Update) 是一种高效的并发同步机制，广泛应用于 Linux 内核等高并发场景。RCU stall 指的是 RCU 机制检测到某些任务长时间没有完成“RCU 读临界区”，导致整个系统的 RCU 回收 (reclaim) 过程被阻塞，进而可能影响系统性能甚至引发卡顿。

问题识别

```
[ 18.532005][C400018] rcu_exp_gp_kthr: sched: RT throttling activated for cpu 4
[ 18.532006][C400018] rcu_exp_gp_kthr: sched: cpu=4, expires=19004000000
now=18528004693 rt_time=952270954 runtime=9500000000 period=1000000000
[ 18.532007][C400018] rcu_exp_gp_kthr: cpu=4, current rcu_exp_gp_kthr (18) is
running for 10233701255 nsec
```

4.6 Boost 问题

在 Android 有的模块会调用 PowerHAL 的接口来调整系统的资源，在 Android 的 log 里面会打印出来下面相关的信息 libPowerHal：

```
06-18 16:02:03.491342...932...999 I libPowerHal: [perfLockAcq] idx:3 hdl:2750 hint:66 pid:932 tid:932 duration:30000 lock_user:vendor.mediatek => ret_hdl:2750
06-18 16:02:03.491364...932...999 I libPowerHal: [setClusterFreq] LAUNCH set cpufreq[0] min:1900000 max:-1 cpufreq[1] min:2800000 max:-1 cpufreq[2] min:2900000 max:-1
06-18 16:02:03.491639...932...999 I libPowerHal: [setClusterFreq] set cpu_ctrl cpufreq: 1900000 -1 2800000 -1 2900000 -1
06-18 16:02:03.492072...932...999 I libPowerHal: [PE] LAUNCH update cmd:1408300, param:100
06-18 16:02:03.492144...932...999 I libPowerHal: [PE] LAUNCH update cmd:300c000, param:15
06-18 16:02:03.492164...932...999 I libPowerHal: [PE] LAUNCH update cmd:1000000, param:0
06-18 16:02:03.495341...932...999 I libPowerHal: [perfSetSysInfo] pid type:24, 3687
06-18 16:02:03.495377...932...999 I libPowerHal: [perfLockRel] hdl:2748, idx:2
06-18 16:02:03.495645...932...999 I libPowerHal: [perfLockAcq] idx:2 hdl:2751 hint:22 pid:932 tid:932 duration:10000 lock_user:vendor.mediatek => ret_hdl:2751
06-18 16:02:03.495657...932...999 I libPowerHal: [setClusterFreq] MTKPOWER_HINT_PACK_SWITCH set cpufreq[0] min:1900000 max:-1 cpufreq[1] min:2800000 max:-1 cpufreq[2] min:2900000 max:-1
06-18 16:02:03.495719...932...999 I libPowerHal: [perfLockRel] hdl:2747, idx:-1
06-18 16:02:03.505185...932...999 I libPowerHal: [perfSetSysInfo] pid type:24, 16098
06-18 16:02:03.544678...932...999 I libPowerHal: [perfLockRel] hdl:2749, idx:1
06-18 16:02:03.544710...932...999 I libPowerHal: [PD] touch_boost update cmd:1404100 param:0
06-18 16:02:03.544754...932...999 I libPowerHal: [PD] touch_boost update cmd:1404300 param:0
06-18 16:02:03.544770...932...999 I libPowerHal: [PD] touch_boost update cmd:1408100 param:0
06-18 16:02:03.548608...932...999 I libPowerHal: cmdSetting - unknown cmd:3409100, scmd:(null), param_1:1
06-18 16:02:03.548635...932...999 I libPowerHal: cmdSetting - unknown cmd:3409200, scmd:(null), param_1:5
```

4.7 Trace 分析

可以参考下面的 FAQ 学习如何看 trace：

<https://online.mediatek.com/apps/quickstart/QS00157>

5 卡顿, 黑屏, 花屏问题

5.1 hwc CMD

- Log CMD:

```
adb shell setprop persist.vendor.debug.hwc.log V && adb shell setprop
vendor.debug.hwc.skip_log 0 && adb shell dumpsys SurfaceFlinger
```

- Dump sf info:

```
adb shell dumpsys SurfaceFlinger > sf.log
```

- Force GPU (GPU 叠图) :

```
adb shell service call SurfaceFlinger 1008 i32 1
```

5.2 显示驱动 CMD

On 都表示开, Off 表示关。

常用的 display mobile log:

```
adb shell "echo mobile:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_mobile_log = 1;
```

如果要开更详细的 log:

```
adb shell "echo detail:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_detail_log = 1;
```

Fence log:

```
adb shell "echo fence:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_fence_log = 1;
```

Irq log:

```
adb shell "echo irq:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_irq_log = 1;
```

抓 display diagnose dump:

```
adb shell "echo diagnose > /sys/kernel/debug/mtkfb && cat /sys/kernel/debug/mtkfb" > mtkfb.txt
```

5.3 Pattern

Pattern 命令需要在 Host OS 执行:

- dsi0 pattern:

```
adb shell "echo gce_wr:1400d178,c61,ffffffff > /sys/kernel/debug/mtkfb"
```

- dsi1 pattern:

```
adb shell "echo gce_wr:1420d178,c61,ffffffff > /sys/kernel/debug/mtkfb"
```

- dp pattern:

```
adb shell "echo gce_wr:1400bf00,41,ffffffff > /sys/kernel/debug/mtkfb"
```

5.4 黑屏

- 背光是否开启
- 如果有 bridge IC, bridge IC 是否 OK
- dsi/dp pattern 是否 OK
- screencap 是否 OK

```
adb shell screencap -d 0/1 /sdcard/1.png
```

如果以上几点都 OK, 最后就需要 display owner 详细看 log 来定位问题。

5.5 花屏/闪屏

- 看 log 中是否有 DISP_OVL/RDMA underflow/abnormal

- size 配置是否 OK

- CMD:

```
adb shell "echo mobile:on > /d/mtkfb"
```

- clk/dram 不够

- clk

- dump:

```
adb shell " echo fmeter > /proc/clkdbg ; cat /proc/clkdbg | grep -e disp"
```

- force 最大:

```
adb shell "echo 0 0 > /sys/module/mtk_mmdvfs_debug/parameters/force_step"
```

- dvfs

- dump:

```
adb shell "cat /sys/kernel/helio-dvfsrc/dvfsrc_dump | grep -e uv -e Mbps"
```

- force 最大:

```
adb shell "echo 0 > /sys/kernel/helio-dvfsrc/dvfsrc_force_vcore_dvfs_opp"
```

- Force GPU 是否 OK

```
adb shell service call SurfaceFlinger 1008 i32 1
```

- dsi/dp pattern 是否 OK

5.6 卡顿

- 看 main log 是否有 fence timeout

```
GED : Frame didn't finished in 1000 ms (try = 1)
GED : Fence fd = 82
GED : fence SurfaceView - #0:2 0
GED : pt timeline-primary-0-13 sw_sync status(0) sync_drv(250117) 0.000000
GED : pt timeline-unknown-0-13 sw_sync status(0) sync_drv(250048) 0.000000
GED : pt timeline-external-3-16 sw_sync status(1) sync_drv(238184) 8203.513127
```

- 看 kernel log 是否有 underflow/abnormal 的 log

- 开启 fence log,看对应 fence 是否 release

```
adb shell "echo fence:on > /d/mtkfb"
```

- 抓 systrace 看耗时位置

```
adb shell perfetto -o /data/misc/perfetto-traces/trace -t 10s sched freq idle am wm gfx
view input
```

display 卡顿相关 trace 的分析 FAQ:

<https://online.mediatek.com/apps/quickstart/QS00295>

5.7 Surfaceflinger Issue Log

- SF watchdog

```
2025-06-02 15:23:00.357 1000 814 814 W surfaceflinger: [SW-WD] <<[composite] >>
timeout, tid=814, spend=3440ms threshold=3000ms
```

- HWC watchdog

```
01-01 00:04:54.882    659    733 W hwcomposer: [SWWatchDog] [SW_WDT] Thread(659) timeout.
id=b400006dea944668 <<[DEV] ioctl(SetCrtc):728>> spend/timeoutThreshold/hangThreshold:
1535/500/10000 ms
01-01 00:04:54.882    659    733 W hwcomposer: [SWWatchDog] [SW_WDT] Thread(659) Name:
composer@3.2-se, State:  D (disk sleep), wait_start:          0.000000,
sleep_start:
```

6 算力优化

6.1 Log 的优化

1. 单个系统中 log 相关的 process 算力占用不要超过 1k，如果占用过大就需要客户推动 log 优化
2. 检查客户是否存在的多个 process 功能重复的记录和存储 log 的问题，如果存在，需要客户组织相关模块进行功能的整合
3. 在 user 版本的 process 的火焰图中看到 print 和 log 相关字眼的 path，要及时找客户确认打印的合理性

6.2 常用优化策略

序号	项目	详细说明
1	debug/test 等 debug 字样的 process 和 flow	在 user 版本上，移除掉 debug 相关的模块
2	fpsgo, frs, gbe, sbe 等相关 process 和 flow	游戏调优相关的模块，理论上在车机上面都可以拿掉
3	Freq, dvfs 等相关的字眼的 process 和 flow	很多的车机客户都是 performance 在跑，遇到调频相关的 process 和 flow 理论上都可以移除掉
4	shell 命令, shell 脚本, adb 命令相关 process	1. 找到 shell 命令是谁触发的，这种一般是 debug 用的 2. ADB 相关的 process 要求客户不用关注，客户出货的版本一般没有 ADB 口
5	Yocto 中 service PID 很大的 process	Service 的 PID 比较大可能存在 service 一直重启的情况
6	APK 带有 DEBUGGABLE flags	发布的 APK 版本为 debug 的，会增加 art 的解析速度
7	flow 中带有 algo 等算法字样的关键字	带有算法的模块，可以跟客户讨论将算法移动到 APU 的可能
8	kcompactd0 占用比较高	通过 pidstat 找每秒换页次数超过 1000 次的模块
9	Kswapd0 占用比较高	需要检查系统是否存在内存泄露，如果没有就需要进行内存优化
10	egl 相关的 flow 占用比较高	找虚拟机厂商看一下 GPU 虚拟化上是否存在可以优化的点
11	Can service, someip, dds 相关的 flow	跟客户一起看一下车身信号相关的 flow 是否可以优化

序号	项目	详细说明
12	review 各个 process 的作用	通过 review process 的作用，来找到 user 版本可以不需要的 process

6.3 异常中断的算力占用

中断的异常使用，不仅会带来中断本身的上半部处理总时间变长，也会带来虚拟机中的中断分发逻辑时间变长间接导致 hypervisor 的算力使用增加。对于中断的算力的优化，我们目前主要集中在异常中断的优化和 timer 中断的优化两部分进行。

1. 异常中断优化

我们可以通过 perfmaster 里面的中断的算力分布，来找到消耗算力排名靠前的中断，跟 owner 一起 review 一下中断的处理是否合理，优化中断上半部中不合理的 flow。

2. Timer 中断优化

- (1) 加上 timer 中断相关的 event：hrtimer_init, hrtimer_start,hrtimer_expire_entry, hrtimer_expire_exit,hrtimer_cancel，并且抓取 3s 的 ftrace log
- (2) 使用 timer 中断统计的脚本，计算处理各个模块使用 timer 的情况
- (3) 跟排名靠前的模块 owner 一起，review 一下其模块是否有不合理的定时器操作

6.4 多维观察法

拿到客户的一个算力占用比较大的 process，我们可以通过多个维度来拆解可以优化的点，下面是可以查看的 process 的一些基本信息：

序号	项目	说明
1	Perfmaster 算力解析表格	找到 process 中算力都是由哪些 thread 贡献的
2	Perfetto/systrace 文件	尽量打开 gfx, binder_driver 等 trace event，了解 process 的显示关系和通讯关系
3	Perf 火焰图	了解 process 的运行 flow，看一下是否存在可以优化的 flow

序号	项目	说明
4	Log 文件	看一下 process 打印的 log, 看一下是否有异常 Log 打印
5	Pidstat 结果	可以看到 process 的上下文和内存切换情况
6	Strace 文件	观察 process 的 syscall 的使用情况

6.5 联动观察法

系统的 process 的执行往往不是一维的，我们可以通过观察某个 process 的启动前后，其它 process 的算力变化，来找到可能优化的点。

举例来说：我们唤醒语音之后，发现 system server，surfaceflinger，HWC，audio 等模块的算力也会随着增加。这样来看我们在优化语音的时候，就不能单看语音这一个 process，还要看一下被其影响的 process 的算力增加是否存在可以优化的点。

在联动观察法中，我们也可以通过 freeze 某个 process，来观察 freeze process 前后系统其它 process 的变化情况：

- (1) 找到 process 的 PID
- (2) 将系统切到: cd /sys/fs/cgroup 目录
- (3) 在该目录下查找 PID 对应的 cgroup 位置: find ./ -name "*pid*"
- (4) Freeze 住该 process: echo 1 > xxx/xxx/cgroup.freeze

6.6 动静态观察法

车机上面分为台架环境和实车环境，台架环境由于很多车身信号没有接入，会表现出来比实车性能更好的假象。动静态分析，主要目的是观察车辆在开动前后各个 process 的算力变化情况，找到动态环境下 process 可以优化的点，以及车身信号是否存在过分/错误下发和模块是否有过分/错误接收的地方。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。